Project 1 (Individual)

Graded

Student

Ioannis Alexandrides

Total Points

37 / 40 pts

Question 1

Project 1

37 / 40 pts

1.1 **Upload here your "Report.pdf" file only!** 37 / 40 pts

Q1 (a) - Likelihood (5 points)

✔ **+ 5 pts** Excelent:
Log-likelihood fully correct.
Score functions correct.
Profiling of β_0 clearly derived and explained.
Algebra clean and logically structured.

+ 4 pts Very good: Correct derivation but minor clarity or algebraic imprecision.

+ 3 pts Good: Mostly correct but explanation thin or slightly unclear.

+ 1 pt Limited: Partial understanding or noticeable errors.

+ 0 pts Incorrect: Fundamental errors in likelihood or profiling.

Q1 (b) Optimisation and Inference (7 points)

✔ **+ 7 pts** Excellent:
Correct optimisation setup.
Hessian correctly obtained.
Covariance = inverse Hessian.
Standard errors correctly extracted.
Clear explanation of inference logic.

+ 6 pts Very Good: All mechanics correct but explanation minimal or too long.

+ 4 pts Good: Core correct but minor technical or interpretative weaknesses.

+ 3 pts Limited: Moderate conceptual or implementation errors.

+ 0 pts Incorrect: Major misunderstanding of MLE or Hessian.

Q1 (c) Wald CIs (5 points)

✔ **+ 5 pts** Excellent:
Correct Wald CIs for β_0 and β_1.
Delta method correctly applied.
Hazard interpretation precise and not overstated.
No unjustified asymptotic claims.

+ 4 pts Very Good: Correct computations, interpretation slightly vague.

+ 3 pts Good: Mechanically correct but interpretation shallow.

+ 2 pts Limited: Incorrect delta derivative or conceptual confusion.

+ 0 pts Incorrect inference.

Q1 (d) Simulation and assymptotics (5 points)

✔ **+ 5 pts** Excellent:
Simulation correctly implemented.
Coverage correctly computed.
Monte Carlo SE computed.
Clear link to asymptotic normality and Wald logic.

+ 4 pts Very Good: Correct mechanics but interpretation brief.

+ 3 pts Good: Correct but weak explanation of asymptotics.

+ 2 pts Limited: Partial misunderstanding of coverage behaviour.

+ 0 pts Incorrect: Major simulation or conceptual errors.

Q2 Latent variable Poisson model (18 points)

✔ + 18 pts Excellent: clear derivation, correct IS, strong reasoning

+ 10 pts Very good: mostly correct, minor weaknesses

+ 3 pts Moderate: correct core but shallow reasoning

+ 3 pts Limited

+ 0 pts Incorrect

💬 – 3 pts Excellent work and good presentation.

1.2 **Upload here your "Report.html" file only!** **0** / 0 pts

Adjustment

+ 3 pts Full Marks. Only if selected submission is on the top category!

+ 2 pts Top end of category

+ 1 pt Above mid-category

+ 0 pts Mid-category

– 1 pt Below mid-category

– 2 pts Bottom end of category

+ 0 pts Click here to replace this description.

Category

+ 37 pts Correct implementation of the task. Potentially could be used as a model answer.

+ 32 pts Implementation of the task is mostly as requested. Essentially only relatively minor issues

+ 27 pts Implementation of the task is mostly as requested, although with some minor issues.

+ 22 pts Implementation of the task is mostly as requested, but with some bigger issues.

+ 17 pts Implementation of the task is incomplete, but at least some working code is present

+ 12 pts Implementation of the task is lacking, major elements of the tasks misunderstood or not implemented properly.

+ 7 pts Poor answer

+ 2 pts Very poor answer

+ 0 pts Files are missing.

✔ + 0 pts Reviewed

1.3 **Upload here your "Report.Rmd" file only!** **0** / 0 pts

✔ + 0 pts Reviewed.

1.4 **Upload here your "code.R" file only!** **0** / 0 pts

✔ **+ 0 pts** Reviewed.

Q1 Project 1

40 Points

To submit your project you will need to upload four files from your R project: "Report.pdf", "Report.html", "Report.Rmd" and "code.R". Please upload the appropriate file under each of the following sub-questions.

Q1.1 Upload here your "Report.pdf" file only!
40 Points

Report.pdf Download

Your browser does not support PDF previews. You can download the file instead.

Q1.2 Upload here your "Report.html" file only!
0 Points

report.html Download

1	Large file hidden. You can download it using the button above.

Q1.3 Upload here your "Report.Rmd" file only!
0 Points

report.Rmd

Download

```
---
title: 'Project 1'
author: "Ioannis Alexandrides (s2767276)"
output:
  html_document:
    number_sections: yes
  pdf_document:
    number_sections: yes
header-includes:
  - \newcommand{\bm}[1]{\boldsymbol{#1}}
  - \newcommand{\mat}[1]{\begin{bmatrix}#1\end{bmatrix}}
  - \usepackage{float}
  - \usepackage{caption}
  - \floatplacement{figure}{H}
editor_options:
  markdown:
    wrap: 72
---

```{r setup, include = FALSE}
knitr::opts_chunk$set(
  echo = TRUE,
  eval = TRUE,
  fig.pos = "H"
)
knitr::opts_knit$set(root.dir = getwd())
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(gridExtra))
theme_set(theme_bw())
set.seed(12345L)
```

```{r code=readLines("code.R"), eval=TRUE, echo=FALSE, results='hide'}
# Do not change this code chunk
# Load function definitions
source("code.R")
```

Exponential likelihood regression model

Log-likelihood and score functions

```{r load-data, echo=TRUE}
dat <- read.csv("expdata.csv")
y   <- dat$y
x   <- dat$x
cat("n =", length(y), "\n")
summary(dat)
```

The model assumes $Y_i \sim \text{Exponential}(\lambda_i)$ where
$\lambda_i = \exp(\beta_0 + \beta_1 x_i)$.

```

**Step 1: Log-likelihood.** Since the density is
$f(y_i \mid \lambda_i) = \lambda_i \exp(-\lambda_i y_i)$, the
log-likelihood for $n$ independent observations is $$
\ell(\beta_0, \beta_1) = \sum_{i=1}^{n}\bigl[\log\lambda_i - \lambda_i y_i\bigr]
= n\beta_0 + \beta_1\sum_{i=1}^{n} x_i - e^{\beta_0}\sum_{i=1}^{n} y_i e^{\beta_1 x_i}.
$$

**Step 2: Score functions.** Differentiating with respect to each
parameter: $$
\frac{\partial\ell}{\partial\beta_0} = n - e^{\beta_0}\sum_{i=1}^{n} y_i e^{\beta_1 x_i},
\qquad
\frac{\partial\ell}{\partial\beta_1} = \sum_{i=1}^{n} x_i - e^{\beta_0}\sum_{i=1}^{n} x_i y_i e^{\beta_1 x_i}.
$$

**Step 3: Closed-form** $\hat\beta_0(\beta_1)$. Setting the first score
to zero and solving: $$
n = e^{\beta_0}\sum_{i=1}^{n} y_i e^{\beta_1 x_i}
\implies
\hat\beta_0(\beta_1) = \log n - \log\!\left(\sum_{i=1}^{n} y_i e^{\beta_1 x_i}\right).
$$

**Step 4: Profile log-likelihood.** Since the first score equation can
be solved explicitly for $\beta_0$ as a function of $\beta_1$, any joint
maximum $(\hat\beta_0, \hat\beta_1)$ must satisfy this relationship.
Substituting $\hat\beta_0(\beta_1)$ into $\ell(\beta_0, \beta_1)$ yields
the profile log-likelihood $$
\ell_p(\beta_1) = \ell\!\left(\hat\beta_0(\beta_1),\, \beta_1\right),
$$ which depends only on $\beta_1$. Maximising over $(\beta_0, \beta_1)$
is therefore equivalent to maximising $\ell_p(\beta_1)$ over $\beta_1$
alone. Once $\hat\beta_1$ is obtained, $\hat\beta_0$ follows immediately
without further numerical optimisation.

## MLE via `optim()`

The function `negloglik_expreg(par, y, x)` computes the negative
log-likelihood $-\ell(\beta_0, \beta_1)$.\
The function `fit_exp_reg(y, x)` then minimises this using `optim()`
with the BFGS method and `hessian = TRUE`, so the Hessian matrix is
returned at the optimum. The covariance matrix is obtained as

$$
\hat{V} = \bigl[-\nabla^2\ell(\hat\beta)\bigr]^{-1},
$$

and the standard errors are the square roots of its diagonal entries.

```{r fit-mle, echo=TRUE}
mle_fit <- fit_exp_reg(y, x)
cat("Estimates:\n");          print(round(mle_fit$estimates, 4))
cat("\nHessian:\n");          print(round(mle_fit$hessian,   4))
cat("\nCovariance matrix:\n"); print(round(mle_fit$vcov,      6))
cat("\nStandard errors:\n");   print(round(mle_fit$se,        4))
```

```{r matrix-display, echo=FALSE, results='asis'}
```

```
H <- mle_fit$hessian
V <- mle_fit$vcov
cat("$$H = \\begin{pmatrix}",
    round(H[1,1],4), "&", round(H[1,2],4), "\\\\",
    round(H[2,1],4), "&", round(H[2,2],4),
    "\\end{pmatrix}$$")
cat("\n\n")
cat("$$V = \\begin{pmatrix}",
    round(V[1,1],4), "&", round(V[1,2],4), "\\\\",
    round(V[2,1],4), "&", round(V[2,2],4),
    "\\end{pmatrix}$$")
```

The profile log-likelihood is plotted below to inspect the shape of the objective function in one dimension.

```{r profile-plot, echo=FALSE, fig.cap="Profile log-likelihood for beta1, with beta0 analytically concentrated out. The red dashed line marks the MLE.", fig.width=6, fig.height=4}
beta1_seq <- seq(mle_fit$estimates["beta1"] - 1.5,
                 mle_fit$estimates["beta1"] + 1.5,
                 length.out = 400)
ll_seq <- sapply(beta1_seq, loglik_profile, y = y, x = x)

ggplot(data.frame(beta1 = beta1_seq, ll = ll_seq), aes(beta1, ll)) +
  geom_line(linewidth = 0.9) +
  geom_vline(xintercept = mle_fit$estimates["beta1"],
             colour = "red", linetype = "dashed", linewidth = 0.8) +
  labs(x = expression(beta[1]),
       y = expression(l[p](beta[1])),
       title = expression(paste("Profile log-likelihood for ", beta[1])))
```

The profile log-likelihood is smooth and strictly concave in a neighbourhood of $\hat\beta_1$, indicating that the MLE is unique. Near the peak, the curve is approximately quadratic, which is consistent with the normal approximation to the distribution of the MLE that underlies Wald inference.

Wald confidence intervals

The function `CI(y, x, alpha)` computes Wald confidence intervals for β_0, β_1, e^{β_1}, and $e^{-\beta_1}$. Used Direct Wald intervals for the regression coefficients. Lastly the delta method is applied for the transformed parameters: for a smooth function $g(\beta)$,

$$
\operatorname{Var}\{g(\hat\beta)\}
\approx
\nabla g(\hat\beta)^\top \hat{V}\, \nabla g(\hat\beta).
$$

For $g(\beta) = e^{\beta_1}$ the gradient is $(0,\, e^{\beta_1})^\top$, so

$$
\widehat{\operatorname{se}}(e^{\hat\beta_1})
= e^{\hat\beta_1}\,\widehat{\operatorname{se}}(\hat\beta_1).
$$

For $e^{-\beta_1}$ the gradient is $(0,\,-e^{-\beta_1})^\top$, giving
the same magnitude scaled by $e^{-\hat\beta_1}$.

```{r wald-ci, echo=TRUE}
ci_table <- CI(y, x, alpha = 0.05)
knitr::kable(ci_table, digits = 4,
             caption = "95% Wald confidence intervals")
```

Interpretation. All three quantities can be interpreted directly in
terms of the hazard.

- β_1 represents the log-hazard ratio per unit increase in x.
  A positive estimate indicates that higher covariate values are
  associated with a higher event rate.

- e^{β_1} is the hazard ratio itself. It gives the
  multiplicative change in the hazard when x increases by one unit.
  If the 95% confidence interval excludes 1, this suggests a
  statistically significant effect of the covariate.

- $e^{-\beta_1}$ can be regarded as the ratio of mean waiting times.
  Since $E[Y_i] = \lambda_i^{-1}$, a one-unit increase in x
  multiplies the expected waiting time by $e^{-\beta_1}$. Values below
  1 indicate shorter expected waiting times at higher covariate
  levels.

Coverage study

The function `estimate_coverage` estimates the empirical coverage of the
95% Wald confidence interval for β_1 using simulation. For each of
$N = 10000$ replicates, n covariate values are resampled with
replacement from the observed x, and responses are generated from the
exponential regression model at the true parameter values. The Wald CI
is then computed and checked to see the coverage of β_1.

The MLE from the full dataset is used as the true parameter vector for
the simulation. It evaluates the finite-sample performance at values
which are consistent with the observed data.

```{r coverage-study, echo=TRUE, cache=TRUE}
b0_true <- mle_fit$estimates["beta0"]
b1_true <- mle_fit$estimates["beta1"]
ns      <- c(30, 50, 100, 200, 500, 1000)

cov_df <- data.frame(n = ns)
cov_df$coverage <- sapply(ns, function(n_i) {
  estimate_coverage(n_i, b0_true, b1_true, x,
                    N = 10000, seed = 12345L)
})
cov_df$mc_se <- sqrt(cov_df$coverage * (1 - cov_df$coverage) / 10000)
```

```
knitr::kable(cov_df, digits = 4,
             caption = "Empirical coverage and Monte Carlo SE")
```

```{r coverage-plot, echo=FALSE, fig.cap="The empirical coverage of the 95% Wald CI for beta1 across sample sizes. The error bars show +/- 1.96 Monte Carlo SE. Lastly the red dashed line marks the nominal 95% level.", fig.width=6, fig.height=4}
ggplot(cov_df, aes(x = n, y = coverage)) +
  geom_line() +
  geom_point(size = 2) +
  geom_errorbar(aes(ymin = coverage - 1.96 * mc_se,
                    ymax = coverage + 1.96 * mc_se), width = 0.04) +
  geom_hline(yintercept = 0.95, colour = "red",
             linetype = "dashed", linewidth = 0.8) +
  scale_x_log10(breaks = ns) +
  labs(x = "Sample size n (log scale)",
       y = "Empirical coverage",
       title = expression(paste("Wald CI coverage for ", beta[1])))
```

Large-sample behaviour. Under standard regularity conditions the MLE satisfies

$$
\sqrt{n}(\hat\beta - \beta)
\xrightarrow{d}
N\!\left(0,\, I(\beta)^{-1}\right),
$$

where $I(\beta)$ is the Fisher information matrix. This asymptotic normality in the results justifies the Wald intervals for large n. For smaller sample sizes, the log-likelihood may be somewhat skewed, and the Wald interval — which is symmetric about $\hat\beta_1$ — can therefore under- or over-cover. As n increases, the quadratic approximation becomes more accurate and the empirical coverage moves closer to the nominal 95% level, as seen in the simulation results.

Monte Carlo standard error. With $N = 10000$ replicates, the Monte Carlo standard error is at most $\sqrt{0.25/10000} = 0.005$. This means that any differences in coverage across sample sizes are due to genuine finite-sample effects rather than simulation variability.

Latent variable Poisson model

Monte Carlo approximation

Derivation of the MC estimator

The model is $X_i \sim N(0, \sigma^2)$ and $Y_i \mid X_i = x \sim \text{Poisson}(e^{\mu + x})$, where all observations are independent. By the law of total expectation,

$$
p(m \mid \mu, \sigma)
= E_X\!\left[\Pr(Y = m \mid X)\right]
= E_X\!\left[\texttt{dpois}(m,\, e^{\mu + X})\right].
```
```

$$

This expectation can be estimated by drawing
$X^{(1)}, \ldots, X^{(K)} \overset{\text{iid}}{\sim} N(0, \sigma^2)$ and
computing

$$
\hat{p}_K(m)
= \frac{1}{K} \sum_{k=1}^{K}
  \texttt{dpois}\!\left(m,\, e^{\mu + X^{(k)}}\right).
$$

Consistency. Each term lies in $[0,1]$, so the summands are i.i.d.
with finite expectation $p(m \mid \mu, \sigma)$. The Strong Law of Large
Numbers gives
$\hat{p}_K(m) \xrightarrow{\text{a.s.}} p(m \mid \mu, \sigma)$ as
$K \to \infty$.

The marginal mean of Y follows from the log-normal moment formula
$E[e^X] = e^{\sigma^2/2}$ for $X \sim N(0, \sigma^2)$:

$$
E[Y] = E\!\left[e^{\mu + X}\right] = e^{\mu + \sigma^2/2}.
$$

Lastly, The Monte Carlo standard error is estimated as
$$\widehat{\text{se}}(m) = \frac{\text{sd}(Z^{(1)}, \ldots, Z^{(K)})}{\sqrt{K}}$$
where $Z^{(k)} = \texttt{dpois}(m, e^{\mu + X^{(k)}})$, following
directly from the standard error of a sample mean.

PMF estimation and comparison with Poisson

```{r latent-pmf, echo=TRUE}
mu_val    <- 0.2
sigma_val <- 0.8
m_vals    <- 0:20

pmf_mc <- dlatentpois_mc(m_vals, mu = mu_val, sigma = sigma_val, K = 10000)
mean_Y <- exp(mu_val + sigma_val^2 / 2)
cat("Marginal mean E[Y] =", round(mean_Y, 4), "\n")
head(pmf_mc)
```

The marginal variance is derived by using the law of total variance:

$$
\operatorname{Var}(Y)
= E\!\left[\operatorname{Var}(Y \mid X)\right]
+ \operatorname{Var}\!\left(E[Y \mid X]\right)
= E[\lambda(X)] + \operatorname{Var}(\lambda(X)).
$$

Since $\lambda(X) = e^{\mu + X}$ is log-normally distributed,

$$
\operatorname{Var}(Y)

```
= e^{\mu + \sigma^2/2}
+ e^{2\mu + \sigma^2}(e^{\sigma^2} - 1),
$$

which exceeds the mean for any $\sigma^2 > 0$. The model therefore
displays overdispersion relative to a Poisson distribution with the same
mean.

```{r pmf-plot, echo=FALSE, fig.cap="Marginal pmf of the latent Poisson model (K = 10000) compared with a Poisson having the same mean.", fig.width=7, fig.height=4, fig.pos="H"}
plot_df <- bind_rows(
  data.frame(m = pmf_mc$m, pmf = pmf_mc$pmf,
             Model = "Latent Poisson (MC)"),
  data.frame(m = m_vals, pmf = dpois(m_vals, lambda = mean_Y),
             Model = "Poisson (same mean)")
)
ggplot(plot_df, aes(x = m, y = pmf, fill = Model)) +
  geom_col(position = "dodge", alpha = 0.85) +
  labs(x = "m", y = "P(Y = m)", fill = "",
       title = expression(paste(
         "Latent Poisson pmf: ", mu, " = 0.2, ", sigma, " = 0.8"))) +
  theme(legend.position = "top")
```

The plot confirms that the latent Poisson assigns more probability to
extreme counts in both tails, on both small and large counts, compared
to the simple Poisson. This reflects the overdispersion induced by the
latent Gaussian variability. This direct consequence of the
overdispersion shown analytically above, illustrates why a Poisson model
would be misspecified for data generated by this hierarchical structure.

Monte Carlo standard errors for different K

```{r pmf-and-se-plots, echo=FALSE, fig.cap="Top: Latent Poisson pmf vs Ordinary Poisson across K. Bottom: MC standard errors across K. Both confirm convergence as K increases.", fig.width=9, fig.height=7, fig.pos="H"}
suppressPackageStartupMessages(library(gridExtra))
Ks      <- c(1e3, 1e4, 1e5)
ord_pois <- dpois(m_vals, lambda = mean_Y)

all_df <- do.call(rbind, lapply(Ks, function(K_i) {
  res <- dlatentpois_mc(m_vals, mu = mu_val, sigma = sigma_val, K = K_i)
  data.frame(m = res$m, pmf = res$pmf, se = res$se,
             ordinary = ord_pois,
             K = paste0("K = 1e", log10(K_i)))
}))

p1 <- ggplot(all_df, aes(x = m)) +
  geom_col(aes(y = pmf), fill = "steelblue", alpha = 0.7) +
  geom_line(aes(y = ordinary), colour = "red") +
  geom_point(aes(y = ordinary), colour = "red", size = 1.2) +
  facet_wrap(~K) +
  labs(x = "m", y = "Probability Mass",
       title = "Latent Poisson vs Ordinary Poisson across K")

```
```

```
p2 <- ggplot(all_df, aes(x = m, y = se, colour = K, group = K)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.5) +
  labs(x = "m", y = "MC Standard Error",
       title = "MC SE decreases as K increases", colour = "K")

grid.arrange(p1, p2, nrow = 2)
```

The Monte Carlo standard error decreases at the expected rate $1/\sqrt{K}$, so increasing K by a factor of 10 reduces the error by $\sqrt{10} \approx 3.16$, in line with the Central Limit Theorem. The SE peaks near the mode because the Poisson probability is the largest and thus the most variable there across Monte Carlo draws. SE is smallest in the tails where most draws contribute near-zero values.

Importance sampling approximation

Importance sampling estimator

The marginal pmf is an expectation under $X \sim N(0, \sigma^2)$. Using the proposal $q(x) = N(a, \sigma^2)$, the importance weights are

$$
w(x) = \frac{\phi(x;\, 0,\, \sigma^2)}{\phi(x;\, a,\, \sigma^2)},
$$

and the importance sampling estimator is

$$
\hat{p}^{\,\text{IS}}(m)
= \frac{1}{K} \sum_{k=1}^{K}
  \texttt{dpois}\!\left(m,\, e^{\mu + X^{(k)}}\right) w(X^{(k)}),
\quad X^{(k)} \sim N(a, \sigma^2).
$$

The weights are computed on the log scale for numerical stability, thus avoiding underflow when evaluating normal densities far from zero. The function `dlatentpois_is(m, mu, sigma, a, K)` implements this estimator.

The Monte Carlo standard error is
$$\widehat{\text{se}}(m) = \frac{\text{sd}(Z^{(1)}, \ldots, Z^{(K)})}{\sqrt{K}}$$
where $Z^{(k)} = \texttt{dpois}(m, e^{\mu + X^{(k)}}) \, w(X^{(k)})$.

Comparison with crude MC for m = 10

```{r is-comparison, echo=TRUE}
K_val    <- 5000
a_values <- c(-1, 0, 1, log(10) - mu_val)

comparison_df <- do.call(rbind, lapply(a_values, function(a_i) {
  res <- dlatentpois_is(10, mu = mu_val, sigma = sigma_val,
                        a = a_i, K = K_val)
  res$a <- round(a_i, 3)
  res
}))
```

```
print(comparison_df, digits = 6)

cat("\nCrude MC (a=0) SE:         ", round(comparison_df$se[2], 6))
cat("\nOptimal IS SE (a=log10-mu): ", round(comparison_df$se[4], 6))
cat("\nVariance reduction factor:  ",
    round((comparison_df$se[2] / comparison_df$se[4])^2, 2), "\n")
```

The table reports results for four choices of the proposal mean a. When $a = 0$, the proposal distribution coincides with the target, so the method reduces to crude Monte Carlo. As a increases toward $\log(10) - \mu$, the proposal centres on the region where $\texttt{dpois}(10, e^{\mu+x})$ is largest, and the Monte Carlo SE falls substantially. The value $a = \log(10) - \mu$ is optimal because it aligns the proposal mean with the value of x at which the Poisson rate equals $m = 10$, maximising overlap between the proposal and the integrand. The point estimates are stable across all values of a, confirming that the estimator remains unbiased regardless of the proposal. The variance reduction factor summarises the efficiency gain compared with crude Monte Carlo.

Session information

```{r session-info, echo=FALSE}
sessionInfo()
```

Code appendix

```{r code=readLines("code.R"), eval=FALSE, echo=TRUE}
# Do not change this code chunk
```

Q1.4 Upload here your "code.R" file only!
0 Points

code.R Download

```r
# Ioannis Alexandrides, s2767276
# Add all your function definitions to this file.
# Each function should be documented using the same style as below.


# Exercise 1: Exponential likelihood regression model


# 1(a) & 1(b): Negative log-likelihood and MLE

#' Negative log-likelihood for exponential regression
#'
#' Computes the negative log-likelihood for the exponential regression model
#' y_i ~ Exponential(rate = exp(beta0 + beta1 * x_i)).
#' We return the negative because optim() minimises by default.
#'
#' @param par numeric vector of length 2: c(beta0, beta1)
#' @param y   numeric vector of strictly positive responses
#' @param x   numeric vector of covariates (same length as y)
#' @return numeric scalar: the negative log-likelihood value
negloglik_expreg <- function(par, y, x) {
  beta0  <- par[1]
  beta1  <- par[2]
  lambda <- exp(beta0 + beta1 * x)
  -sum(dexp(y, rate = lambda, log = TRUE))
}

#' Fit exponential regression model by maximum likelihood
#'
#' Minimises the negative log-likelihood using optim() with BFGS.
#' The covariance matrix is the inverse of the observed Hessian at the MLE,
#' and standard errors are the square roots of its diagonal entries.
#'
#' @param y numeric vector of strictly positive responses
#' @param x numeric vector of covariates (same length as y)
#' @return list with components: estimates, hessian, vcov, se
fit_exp_reg <- function(y, x) {
  # Initialise beta0 using log(1/ybar) = -log(ybar), beta1 = 0
  init <- c(beta0 = -log(mean(y)), beta1 = 0)
  fit <- optim(
    par     = init,
    fn      = negloglik_expreg,
    y       = y,
    x       = x,
    method  = "BFGS",
    hessian = TRUE
  )
  vcov_mat  <- solve(fit$hessian)
  se        <- sqrt(diag(vcov_mat))
  names(se) <- c("beta0", "beta1")
  list(
    estimates = setNames(fit$par, c("beta0", "beta1")),
    hessian   = fit$hessian,
```

```r
    vcov     = vcov_mat,
    se       = se
  )
}

# 1(b): Profile log-likelihood (used for the plot in part b)

#' Profile estimate of beta0 as a function of beta1
#'
#' Closed-form solution from setting the first score equation to zero.
#' This lets us reduce the 2D optimisation to 1D.
#'
#' @param beta1 numeric scalar
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @return numeric scalar: profile MLE of beta0
beta0_profile <- function(beta1, y, x) {
  log(length(y)) - log(sum(y * exp(beta1 * x)))
}

#' Profile log-likelihood as a function of beta1
#'
#' Substitutes the closed-form beta0_profile(beta1, y, x) into the
#' log-likelihood, giving a function of beta1 alone.
#'
#' @param beta1 numeric scalar
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @return numeric scalar: profile log-likelihood value
loglik_profile <- function(beta1, y, x) {
  b0 <- beta0_profile(beta1, y, x)
  -negloglik_expreg(c(b0, beta1), y, x)
}

# 1(c): Wald confidence intervals

#' Wald confidence intervals for exponential regression parameters
#'
#' Computes Wald CIs for beta0, beta1, exp(beta1) and exp(-beta1).
#' Direct Wald intervals for the regression coefficients; delta method
#' for the two transformations.
#'
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @param alpha numeric significance level, default 0.05
#' @return data frame with columns: param, hat, se, lo, hi
CI <- function(y, x, alpha = 0.05) {
  mle <- fit_exp_reg(y, x)
  b   <- mle$estimates
  V   <- mle$vcov
  z   <- qnorm(1 - alpha / 2)

  # Wald CIs for beta0 and beta1
  rows <- data.frame(
    param = c("beta0", "beta1"),
    hat   = as.numeric(b),
```

```r
    se    = as.numeric(mle$se),
    lo    = as.numeric(b) - z * as.numeric(mle$se),
    hi    = as.numeric(b) + z * as.numeric(mle$se),
    row.names = NULL
  )

  # Delta method for exp(beta1): gradient is (0, exp(beta1))
  eb1    <- exp(b["beta1"])
  g1     <- c(0, eb1)
  se_eb1 <- sqrt(as.numeric(t(g1) %*% V %*% g1))
  rows   <- rbind(rows, data.frame(
    param = "exp(beta1)",
    hat   = as.numeric(eb1),
    se    = se_eb1,
    lo    = as.numeric(eb1) - z * se_eb1,
    hi    = as.numeric(eb1) + z * se_eb1
  ))

  # Delta method for exp(-beta1): gradient is (0, -exp(-beta1))
  emb1    <- exp(-b["beta1"])
  g2      <- c(0, -emb1)
  se_emb1 <- sqrt(as.numeric(t(g2) %*% V %*% g2))
  rows    <- rbind(rows, data.frame(
    param = "exp(-beta1)",
    hat   = as.numeric(emb1),
    se    = se_emb1,
    lo    = as.numeric(emb1) - z * se_emb1,
    hi    = as.numeric(emb1) + z * se_emb1
  ))

  rows
}

# 1(d): Coverage simulation

#' Empirical coverage probability of the Wald CI for beta1
#'
#' Simulates N datasets of size n, fits the model to each, and checks
#' whether the Wald CI for beta1 covers the true value. Covariates are
#' resampled with replacement from the observed x vector.
#'
#' @param n     integer sample size for each simulated dataset
#' @param beta0 numeric true intercept
#' @param beta1 numeric true slope
#' @param x     numeric covariate pool to resample from
#' @param N     integer number of simulation replicates, default 10000
#' @param alpha numeric significance level for CI, default 0.05
#' @param seed  integer optional random seed for reproducibility
#' @return numeric scalar: empirical coverage probability
estimate_coverage <- function(n, beta0, beta1, x, N = 10000,
                              alpha = 0.05, seed = NULL) {
  if (!is.null(seed)) set.seed(seed)
  z       <- qnorm(1 - alpha / 2)
  covered <- logical(N)
  for (i in seq_len(N)) {
    xi     <- sample(x, size = n, replace = TRUE)
```

```r
    lambda <- exp(beta0 + beta1 * xi)
    yi     <- rexp(n, rate = lambda)
    # tryCatch handles rare numerical failures gracefully
    fit    <- tryCatch(fit_exp_reg(yi, xi), error = function(e) NULL)
    if (is.null(fit)) next
    b1_hat     <- fit$estimates["beta1"]
    se_b1      <- fit$se["beta1"]
    covered[i] <- (b1_hat - z * se_b1 <= beta1) &&
      (beta1  <= b1_hat + z * se_b1)
  }
  mean(covered)
}


# Exercise 2: Latent variable Poisson model


# 2.1(a) & 2.1(b): Monte Carlo estimator

#' Monte Carlo estimator of the marginal pmf of the latent Poisson model
#'
#' Model: X ~ N(0, sigma^2), Y|X ~ Poisson(exp(mu + X)).
#' Estimates p(m | mu, sigma) by averaging dpois(m, exp(mu + X)) over
#' K draws from the latent distribution. Consistent by the SLLN.
#'
#' @param m     integer vector of count values to evaluate the pmf at
#' @param mu    numeric scalar mean parameter
#' @param sigma numeric scalar standard deviation, sigma > 0
#' @param K     integer number of Monte Carlo samples
#' @return data frame with columns: m, pmf, se
dlatentpois_mc <- function(m, mu, sigma, K) {
  X   <- rnorm(K, mean = 0, sd = sigma)
  lam <- exp(mu + X)
  out <- lapply(m, function(m_j) {
    h <- dpois(m_j, lambda = lam)
    data.frame(m = m_j, pmf = mean(h), se = sd(h) / sqrt(K))
  })
  do.call(rbind, out)
}

# 2.2(a) & 2.2(b): Importance sampling estimator

#' Importance sampling estimator of the marginal pmf of the latent Poisson model
#'
#' Uses proposal q(x) = N(a, sigma^2) instead of the target N(0, sigma^2).
#' Importance weights w(x) = phi(x; 0, sigma^2) / phi(x; a, sigma^2) correct
#' for the distributional shift. Weights computed on the log scale to avoid
#' numerical underflow for large shifts in a.
#'
#' @param m     integer vector of count values to evaluate the pmf at
#' @param mu    numeric scalar mean parameter
#' @param sigma numeric scalar standard deviation, sigma > 0
#' @param a     numeric scalar proposal mean
#' @param K     integer number of importance samples
#' @return data frame with columns: m, pmf, se
dlatentpois_is <- function(m, mu, sigma, a, K) {
```

```
  X_q   <- rnorm(K, mean = a, sd = sigma)
  # Log-scale weights to avoid underflow
  log_w <- dnorm(X_q, mean = 0, sd = sigma, log = TRUE) -
    dnorm(X_q, mean = a, sd = sigma, log = TRUE)
  w     <- exp(log_w)
  lam   <- exp(mu + X_q)
  out <- lapply(m, function(m_j) {
    hw <- dpois(m_j, lambda = lam) * w
    data.frame(m = m_j, pmf = mean(hw), se = sd(hw) / sqrt(K))
  })
  do.call(rbind, out)
}


```